FORMAL LANGUAGES AND COMPILERS

prof.s Giovanni Agosta, Luca Breveglieri and Daniele Cattaneo

Exam of THU 5 JUNE 2025 – Theory Part

LAST NAME (SURNAME) + FIRST NAME:

(capital letters please)

MATRICOLA: SIGNATURE:

(or PERSON CODE)

TEACHER: ☐ Prof. G. AGOSTA – ☐ Prof. L. BREVEGLIERI – ☐ Prof. D. CATTANEO

INSTRUCTIONS – READ CAREFULLY:

- **The exam is open book: textbooks and personal notes are permitted.**
- **Pencil writing is permitted, except on the cover page (this sheet).**
- **Please write in the free space left and if necessary continue on the back side of the sheet; do not attach new sheets and do not replace the existing ones.**
- **Time:** lab part 1 h – **theory part 2 h.**

SCORING AND GRADE ATTRIBUTION:

- The exam is in written form and consists of two parts:

Theory	syntax and semantics of languages, divided in four sections:	(75% of the final grade)
1)	regular expressions and finite automata	(1 exercise × 10 points)
2)	syntax analysis and parsing methodologies	(1 exercise × 10 points)
3)	grammar design – translation – semantic analysis	(2 exercises × 5 points)
4)	*bonus* question	(1 exercise × 5 points)
Lab	compiler design by Flex and Bison	(25% of the final grade)
1)	basic questions	(30 points)
2)	*bonus* question	(5 points)

- For the theory part to be valid, the candidate must achieve a grade of at least 5/10 in each of the three sections 1, 2 and 3. If this is not the case, section 4 is not evaluated. Section 4 has no threshold on its own. Correctly answering all the questions in sections 1, 2 and 3 allows the candidate to achieve a grade of 30/30 (but not *cum laude*) for the theory part.
- For the lab part to be valid, the candidate must achieve a grade of at least 15/30 in the basic questions. The bonus question is evaluated only if the threshold is reached.
- To pass the exam, the candidate must succeed in both parts (theory and lab), in one call or more calls separately, but within one year (12 months) between the two parts.
- The final grade is the weighted average of the theory part (75 %) and of the lab part (25 %), and must be $\geq 18/30$ (*cum laude* $\geq 32/30$).

1 Regular expressions and finite automata (10 points)

Consider the regular expression R below, over the two-letter alphabet $\Sigma = \{\ a,\ b\ \}$

$$R = \left(a^{+} \ \mid \ b\,b \ \mid \ b\,b\,a \right)^{*}$$

Answer the following questions (use the figures / tables / spaces on the next pages):

(a) List all the strings of length ≤ 3 (less than or equal to 3) that are generated by the regular expression R.

(b) Say which strings of those listed at point (a) are ambiguous (if any), and for each of them show two different derivations.

(c) By using the Berry-Sethi (BS) method, build a deterministic automaton A equivalent to the regular expression R.

(d) Determine if the automaton A of point (c) is minimal, and if necessary minimize it.

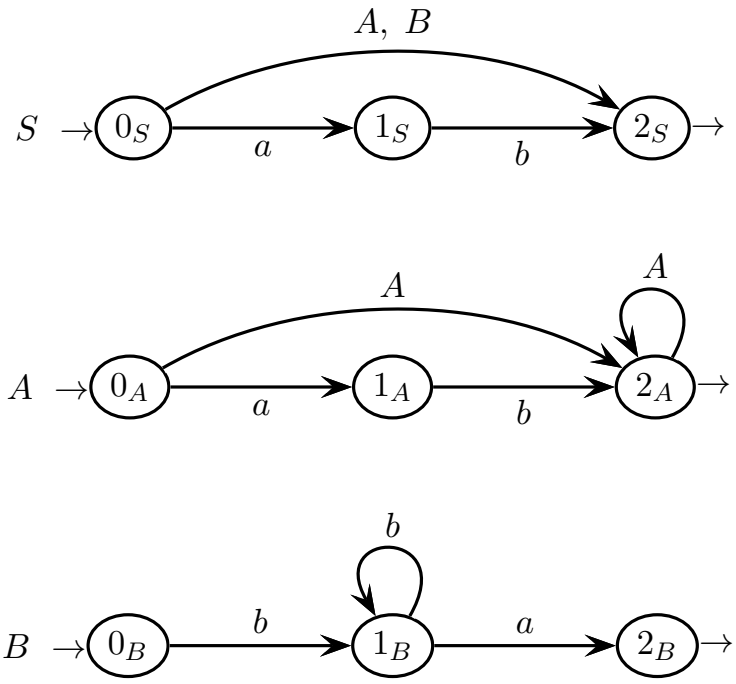
(e) Construct the complement automaton B of the minimal version of automaton A, and verify that automaton B rejects all the strings found at point (a), and accepts the three shortest strings invalid for automaton A. Is the automaton B minimal ? Justify your answer.

(f) By using the Brzozowski-McCluskey (BMC) method, find a regular expression R' equivalent to the complement automaton B.

---

2 Syntax analysis and parsing methodologies (10 points)

Consider the following grammar G, described by a machine net, over the two-letter terminal alphabet $\Sigma = \{ a,\ b \}$ and the three-symbol nonterminal alphabet $V = \{ S,\ A,\ B \}$ (axiom S):



Answer the following questions (use the figures / tables / spaces on the next pages):

(a) Draw at least three different syntax trees of the valid string below:

$$a\,b$$

How many syntax trees does this string admit?

(b) Draw the complete ELR pilot of grammar G (part of it is already given), list all the conflicts (if any), and determine whether grammar G is of type ELR(1) or not. Explain your answer.

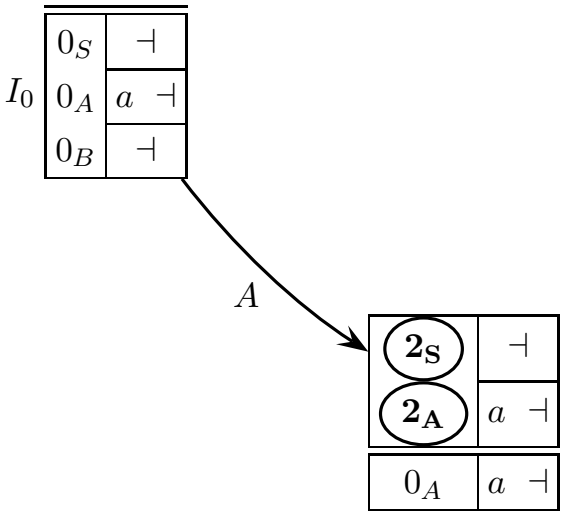
(c) Find all the guide sets of the machine net of grammar G (on the call arcs and exit arrows), list all the conflicts (if any), and determine whether G is of type ELL(1) or not. Explain your answer.

(d) Now consider the language described by machine A alone. Is the language regular or context free? Explain your answer.

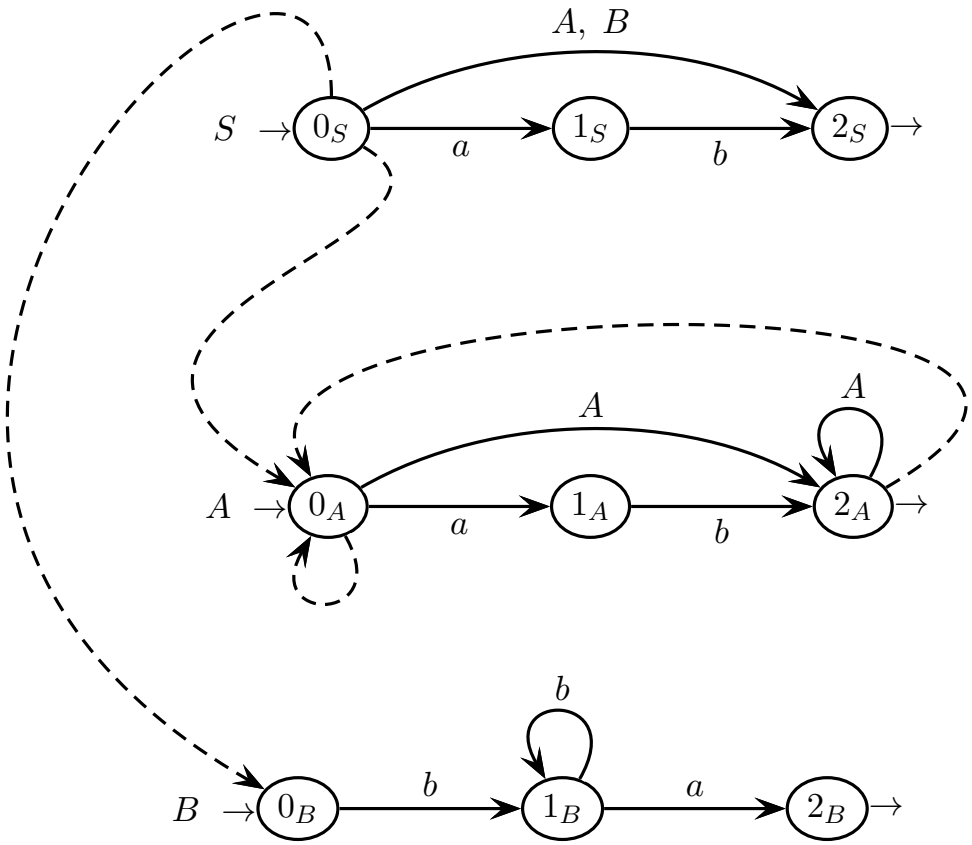
---

syntax trees of string $a\,b$ – question (a)

pilot to be completed and conflict analysis – question (b)



guide sets and conflict analysis – question (c)



language analysis – question (d)

free space for continuation or notes – any question

3 Grammar design – translation – semantic analysis ($5 + 5$ points)

1. Consider the following code fragment in (a slight variation of) the Rulebook language, which describes an implementation of the "tic-tac-toe" game (the comments `//` are not part of the language):

```
coroutine play ( ) -> TicTacToe {              // declaration of coroutine
   frame board : Board
   while ! board.is_full ( ) {
       action mark (Int x, Int y) [            // interruption / resumption point
          x < 3, x >= 0, y < 3, y >= 0, board.get (x, y) == 0      // constraints
       ] { board.set (x, y) }                  // end of the int. / res. point
       if board.three_in_a_line ( ) { return }
       board.change_player ( )
   }
}
```

The Rulebook language allows to declare coroutines, which are functions that can be interrupted and resumed, by prefixing them with the `coroutine` keyword, e.g., the `play` coroutine declared above.

Within a coroutine, the `action` keyword is used to declare an interruption / resumption point, where new values, declared as parameters of the interruption point, can be obtained from the caller, e.g., the `mark` interruption / resumption point within the `play` coroutine above. Such parameters are subjected to constraints (see the example above), expressed as a (possibly empty) list of expressions. For instance, here is a caller function `main` that repeatedly resumes the coroutine `play` at the `mark` point and each time passes parameters:

```
function main ( ) -> Int {
   let game = play ( )
   game.mark (0, 0)      // resume mark and pass parameters 0, 0
   game.mark (1, 0)      // resume mark and pass parameters 1, 0
   game.mark (1, 1)      // ...
   game.mark (2, 0)      // ...
   game.mark (2, 2)      // ...
   return game.board.three_in_a_line ( )
}
```

Assume that expressions and types are modeled by the nonterminals EXPR and TYPE, respectively, and that the identifiers are modeled by the terminal ID; none of them needs to be expanded. Furthermore, the declaration of a coroutine is modeled by the following grammar fragment:

$$\left\{ \begin{array}{l} \langle \text{COROUTINE} \rangle \rightarrow \text{coroutine ID } \text{“(”} \ \langle \text{PARAM\_LST} \rangle \ \text{“)”} \ \text{“}->\text{”} \ \langle \text{TYPE} \rangle \ \text{“\{”} \ \langle \text{BLOCK} \rangle \ \text{“\}”} \\ \hline \langle \text{BLOCK} \rangle \rightarrow \langle \text{STATEMENT} \rangle^{+} \\ \hline \langle \text{STATEMENT} \rangle \rightarrow \langle \text{VAR\_DCL} \rangle \mid \langle \text{INTERRUPT} \rangle \mid \langle \text{WHILE} \rangle \mid \langle \text{COND} \rangle \mid \langle \text{CALL} \rangle \mid \text{return } [\ \langle \text{EXPR} \rangle\ ] \end{array} \right.$$

Answer the following questions (use the figures / tables / spaces on the next pages):

(a) Design the rule for the nonterminal INTERRUPT, which models an interruption / resumption point. Reuse (and implement) the nonterminal PARAM_LST to represent a list of parameters.

(b) Draw the syntax tree for the interruption point `mark` from the "tic-tac-toe" example (start the tree from the nonterminal INTERRUPT).

(c) Design the rules for the nonterminals WHILE and COND, considering that the latter has an optional clause "else".

2. Consider a grammar that represents code in a programming language, where spacing conveys the meaning of scope blocks, e.g., Python. The grammar is defined as follows (axiom S):

$$\begin{cases} 1: & S \rightarrow P \\ 2: & P \rightarrow L\ P \\ 3: & P \rightarrow \varepsilon \\ 4: & L \rightarrow B\ x\ n \\ 5: & L \rightarrow B\ y\ n \\ 6: & B \rightarrow b\ B \\ 7: & B \rightarrow \varepsilon \end{cases}$$

The terminals b, x, y and n respectively represent:

- b: a blank space
- x: a generic statement (not a block header)
- y: the header of a block of statements
- n: a newline

The nonterminal P models a program, defined as a list of lines (nonterminal L), all of which have a prefix (nonterminal B), which consists of a number of blank spaces (terminal b).

The allowed amount of blank spaces b at the beginning of each line is constrained by the specification of the language as follows. A line that contains the header of a block of statements forces the next line in the program to be indented exactly one blank space b to the right more than the block header. The body of a block consists of all the lines with an indentation level, i.e., number of blank spaces b, greater than the indentation level of the block header. Any statement (except the first after a block header) terminates one or more blocks if it has fewer blank spaces than required. That terminating statement is considered outside of the block(s) terminated. Said differently, the statements that are not the first after a block header, must be indented as much as the previous statement, or even less.

Answer the following questions (use the figures / tables / spaces on the next pages):

(a) Design an attribute grammar that checks that the indentation level of each statement is correct, by synthesizing an attribute *valid* that is *true* if the program has a correct indentation, and is *false* otherwise. The following attributes should be employed:

nonterm.	**name**	**type**	**domain**	**meaning**
S, P	*valid*	left	boolean	*true* if the line list has a correct indentation, otherwise *false*
P	*min*	right	integer	minimum allowed indentation for the line
P	*max*	right	integer	maximum allowed indentation for the line
L	*is_hdr*	left	boolean	*true* if the line is a block header, *false* if not
L, B	*count*	left	integer	amount of indentation

(b) Compute the values of the attributes for the syntactically and semantically valid string below:

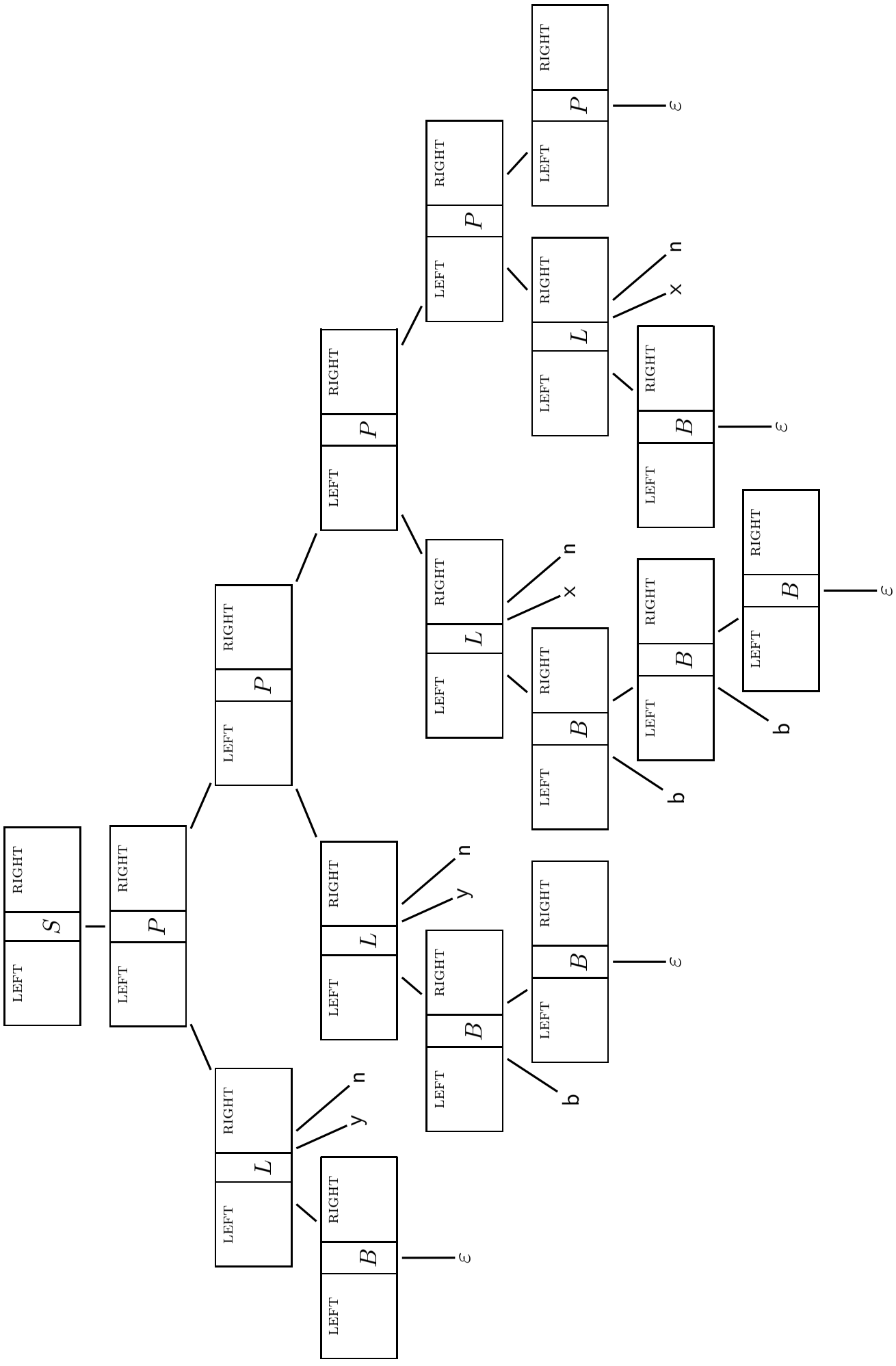
```
y n b y n b b x n x n
```

by decorating the syntax tree on the next pages.

attribute grammar – question (a)

#	*syntax*	*semantics*
1:	$S_0 \rightarrow P_1$	
2:	$P_0 \rightarrow L_1\ P_2$	
3:	$P_0 \rightarrow \varepsilon$	
4:	$L_0 \rightarrow B_1\ x\ n$	
5:	$L_0 \rightarrow B_1\ y\ n$	
6:	$B_0 \rightarrow b\ B_1$	
7:	$B_0 \rightarrow \varepsilon$	

tree of string `y n b y n b b x n x n` decorated with attribute values – question (b)



free space for continuation or notes – any question

4 Bonus question (5 points)

Consider the BNF grammar G below, over the two-letter terminal alphabet $\Sigma = \{\ a,\ b\ \}$ (axiom S):

$$G \begin{cases} S \ \rightarrow \ a\ S\ b\ S \\ S \ \rightarrow \ a\ S \\ S \ \rightarrow \ \varepsilon \end{cases}$$

Answer the following questions (use the figures / tables / spaces on the next pages):

(a) List all the strings of length ≤ 3 (less than or equal to three) that belong to language $L\,(G)$.

(b) Argue that grammar G is ambiguous by exhibiting the two shortest ambiguous strings generated by G, and all their syntax trees.

(c) Find a non-ambiguous BNF grammar G' equivalent to grammar G, and justify your answer.

(d) Is language $L\,(G)$ deterministic ELR or ELL ? Explain your answer.

---

free space for continuation or notes – any question

free space for continuation or notes – any question